

Section A: Scenario-Based Questions

For each scenario, decide Relational or NoSQL (and which NoSQL type, if applicable), then justify your choice in 2- 3 sentences using the concepts from the session (ACID/BASE, schema flexibility, query pattern).

A1. Payment Processing System

A fintech app needs to debit one account and credit another atomically, with strict rollback if either step fails.

#RDBMS #ACID #FIXED-SCHEMA #SQL-QUERY

Relational DB PostgreSQL because it supports ACID properties and has a structured approach where relationships between tables and accurate transactions are fundamental.

A2. E-commerce Product Catalog

Products range from books to electronics to furniture — each category has wildly different attributes (e.g., "page count" vs "battery life" vs "dimensions").

#NoSQL #BASE #FLEXIBLE-SCHEMA #JSON-QUERY

NoSQL or a Document Database (example: MongoDB) because products have different attributes, so a flexible schema is more appropriate. Product data is stored as JSON documents, making it easy to add new fields without modifying the schema.

A3. Social Media Friend Recommendations

The app needs to suggest "people you may know" by finding friends-of-friends up to 3 levels deep.

#NoSQL_Graph Database #Neo4j #FLEXIBLE-SCHEMA #RELATIONSHIP-QUERY

NoSQL (Graph DB like 'Neo4j) because it stores relationships between users. It is optimized for queries like "friends of friends", making recommendations fast and efficient.

A4. IoT Sensor Network

Thousands of sensors report temperature readings every second; the system must handle very high write throughput, and reads are mostly "give me the last 24 hours for sensor X."

#NoSQL #BASE #FLEXIBLE-SCHEMA #QUERY-BY-DEVICE

NoSQL because it can store a large number of sensor readings and scale to handle frequent writes. It follows the BASE model and allows sensor data to be queried by sensor ID and time range, such as retrieving the last 24 hours of readings.

A5. Session/Cart Management

A retail site needs to store each logged-in user's shopping cart for fast retrieval by session ID, with no complex querying required.

#NoSQL #BASE #FLEXIBLE-SCHEMA #KEY-VALUE-QUERY

NoSQL because each shopping cart is stored and retrieved using a unique session ID (the key). It follows the BASE model and is optimized for simple, fast lookups rather than complex SQL queries. Store cart by session ID -> retrieve cart by session ID

A6. Financial Reporting Dashboard

A finance team needs to join customer, order, and payment tables to produce monthly revenue reports with strict accuracy.

#RDBMS #ACID #FIXED-SCHEMA #SQL-JOINS

Relational Database, for example, PostgreSQL because it supports ACID, making financial data accurate and reliable. Its fixed schema and SQL queries make it easy to join customer, order, and payment tables to create monthly revenue reports.

Section B: Multiple Choice Questions

B1. What does a foreign key primarily enforce?

- a) Query performance
- b) Referential integrity between tables**
- c) Schema flexibility
- d) Data compression

B2. Which statement about NoSQL is most accurate?

- a) NoSQL means no schema at all
- b) NoSQL databases can never scale
- c) NoSQL favors flexible schema over strict enforcement**
- d) NoSQL databases don't support any data validation

B3. In the ACID model, "Isolation" refers to:

- a) Data being stored on isolated servers
- b) Concurrent transactions not interfering with each other**
- c) Backups being isolated from production
- d) Tables being isolated from each other

B4. Which NoSQL type is best suited for "friends of friends" traversal queries?

- a) Key-value
- b) Document
- c) Column-family
- d) Graph

B5. BASE stands for:

- a) Basic Availability, Stateless Execution
- b) Basically Available, Soft state, Eventual consistency
- c) Balanced Access, Secure Encryption
- d) Batch Aggregation, Synchronous Execution

B6. A primary key automatically implies:

- a) The column is encrypted
- b) An index is often created for uniqueness enforcement
- c) The table has no foreign keys
- d) The column must be a string type

B7. Which use case is the best fit for a column-family database?

- a) Bank account transfers
- b) Per-user daily time-series metrics with many optional fields
- c) Storing a single user's session token
- d) Modeling friendships between users

B8. What is a key trade-off when choosing BASE over ACID?

- a) You gain strong consistency but lose availability
- b) You gain availability/partition tolerance but accept eventual consistency
- c) You gain faster joins but lose indexing
- d) You gain schema enforcement but lose flexibility

B9. Embedding line items inside an order document (vs. referencing them) is a pattern typical of:

- a) Column-family databases
- b) Graph databases
- c) Document databases
- d) Key-value databases

B10. Which scenario is the weakest fit for a graph database?

- a) Fraud ring detection across connected accounts
- b) Mutual friend discovery
- c) Large-scale tabular financial reporting with millions of rows
- d) Recommendation engines based on relationships

Section C: Theory Questions

C1.

Explain the difference between a primary key and a foreign key, and describe one real-world consequence of missing a foreign key constraint.

A primary key is a unique identifier for each row in a table.

A foreign key links one table to another by referencing the primary key.

C2.

Compare ACID and BASE across consistency, availability, and typical use cases. Give one example use case for each that was not used in the session notes.

ACID supports consistency and is used for applications where data requires accuracy.

BASE prioritizes availability and scalability and accepts eventual consistency.

C3.

"NoSQL means no schema." Explain why this is a common misconception and describe what actually happens to schema in a NoSQL system.

NoSQL means "not only SQL". The misconception derives from the flexible NoSQL schema.

New fields can be added to documents without changing the entire database structure.

C4.

Describe the four main NoSQL types (document, key-value, column-family, graph) in one sentence each, and give one data engineering example per type.

Document database: Stores data as JSON-like documents. (ex: user profiles)

Key-value database: Stores data as key-value pairs for fast lookups. (ex: Shopping carts or user sessions)

Column-family database: Stores large amounts of data by columns and is optimized for high write volumes. ex: Sensor or IoT data.

Graph database: Stores data as nodes and relationships. (ex: Friend suggestions in social media)

C5.

A team wants to migrate a fixed-schema product table to a flexible-schema document model because product attributes vary heavily by category. What is gained, and what risk should the team watch for after migration?

A document database allows different products to have different attributes without changing the database structure. The risk is that product information may become inconsistent if the data is not checked or validated. Data entry formats should be validated in order similar products have the same attributes.

Section D: Practical Questions (Try it yourself)

D1. Relational -Build and Query

Using `sqlite3`, create two tables: `employees(employee_id PK, name, dept_id FK)` and `departments(dept_id PK, dept_name)`. Insert at least 2 departments and 3 employees. Write a query that joins both tables to list each employee's name alongside their department name.

D2. ACID - Transaction Rollback

Using `sqlite3`, create a `wallet(user, balance)` table. Simulate a transfer of 30 units from one user to another inside a transaction. Deliberately raise an exception before committing, then roll back and print the table to confirm balances are unchanged.

D3. Document Model - Variable Attributes

Using Python dictionaries, create a small in-memory "collection" (list of dicts) of at least 3 products from different categories (e.g., book, laptop, chair), where each product has different fields beyond a shared sku and name. Write a function that filters and returns only products that have a `battery_life` field.

D4. Key-Value Store - Simulated Cache

Using a Python dict as a key-value store, write code to store 3 user sessions keyed by "session:<id>", then write a function `get_session(session_id)` that returns the session or None if not found (simulating a cache miss).

D5. Eventual Consistency

Using two Python dicts (`replicaA`, `replicaB`) to simulate replicas, write a value to `replicaA` only, show that reading it from `replicaB` returns nothing (stale/missing), then simulate a sync and show `replicaB` now has the value. Add a short comment explaining which BASE property this demonstrates.

D6. Graph - Friends of Friends

Using an adjacency list (dict of lists) representing a small social graph with at least 4 users, write code to compute the friends-of-friends for one user, excluding the user themselves and their direct friends.